
Temporal Architecture for AI Coding Agent Harnesses

First Author¹, Second Author², Third Author², Fourth Author¹ and Fifth Author¹

*Equal Contribution, ¹Affiliation 1, ²Affiliation 2

AI coding agent harnesses face a central tension: faster iteration improves throughput but degrades output quality. We propose *Verified Iterations per Hour* (VIH), a composite metric that captures this tradeoff, and develop a formal framework grounding temporal architecture in stochastic scheduling, queueing theory, speculative execution, and cache competitive analysis. Our contributions include: (1) an information-theoretic model of context freshness with exponential staleness decay; (2) a speculation ratio test determining when speculative execution has positive expected value; (3) a cache-staleness tradeoff theorem yielding an EOQ-analog optimal refresh interval; (4) a decision-theoretic framework for execution mode selection under asymmetric costs; and (5) a Pareto frontier analysis showing that VIH is maximized at the unit-elasticity point. Illustrative calibration against published data from multiple harnesses suggests that speculative pipelining, tiered verification, and asynchronous governance can reduce cycle time by approximately 34%, while naive persistent state may be VIH-negative when context quality penalties are large.

1. Introduction

The dominant complaint about AI coding agent harnesses is speed. GSD takes 45–60 minutes per full cycle; 24 open issues cite iteration time as the primary bottleneck (Iskohm, 2026). But speed is not merely a user experience concern. In agentic loops, faster iteration enables more attempts per unit time, and empirically, more attempts can compensate for lower per-attempt accuracy up to a threshold (Anonymous, 2025). A model scoring 2% higher but taking 3× longer can be worse for agentic loops (Iskohm, 2026).

The naive response is to make everything faster. This paper argues that the naive response is wrong, or at least incomplete. Raw speed, measured as iterations per hour, is a misleading metric because it ignores the quality of those iterations. Skipping verification to go faster compounds errors geometrically and destroys effective throughput. We propose *Verified Iterations per Hour* (VIH) as a useful intermediate metric: the rate at which an agent completes iterations that pass all verification gates. VIH does not capture all quality dimensions (architectural correctness, maintainability, and failure severity lie outside its scope), but it formalizes the speed–quality tradeoff that dominates current harness design decisions.

This reframing has deep implications for pipeline design, caching policy, speculative execution strategy, and execution mode selection. We develop these implications formally, drawing on four classical bodies of theory:

- (i) **Stochastic scheduling** (Graham, 1966; Möhring et al., 1984; Pinedo, 2016): multi-stage pipelines with probabilistic failure and speculative rollback.
- (ii) **Queueing theory** (Hopp and Spearman, 2000; Kingman, 1961; Little, 1961): bottleneck identification, the utilization cliff, and the throughput–latency tradeoff.
- (iii) **Speculative execution** (Smith, 1981; Tomasulo, 1967; Yeh and Patt, 1991): conditions under which pre-computing likely future results has positive expected value.

- (iv) **Cache competitive analysis** (Belady, 1966; Denning, 1968; Sleator and Tarjan, 1985): staleness–computation tradeoffs and optimal refresh policies.

Our contributions are:

1. A **context fidelity model** (§3) with exponential staleness decay, calibrated against the 17.1% Fresh Eyes advantage (Suzgun and Kalai, 2024).
2. A **VIH decomposition and optimization** (§4) showing that $\text{VIH} = \lambda_{\text{raw}} \cdot p_{\text{verify}}$, with the optimal operating point at the unit-elasticity condition.
3. A **speculation ratio test** (§6) and depth limit for multi-stage pipelines.
4. A **cache-staleness theorem** (§7) yielding an EOQ-analog optimal refresh interval.
5. A **Pareto frontier analysis** (§9) proving that mixing strategies dominate pure strategies in concave frontier regions.
6. **Illustrative calibration** (§10) against published harness data.

Throughout, we distinguish established results applied without modification (marked [E]), novel synthesis of known results in the agent harness setting ([S]), and conjectured claims supported by empirical data but lacking formal proof ([C]).

2. Preliminaries and Definitions

2.1. The Agent Pipeline as a Stochastic DAG

Definition 2.1 (Agent Pipeline DAG). *An agent pipeline is a stochastic DAG $G = (V, E, D, q)$ where $V = \{v_1, \dots, v_n\}$ are pipeline stages, $E \subseteq V \times V$ encodes precedence constraints, each stage v_i has duration $t_i \sim D_i$ with mean μ_i and variance σ_i^2 , and failure probability $q_i \in [0, 1]$ with success probability $p_i = 1 - q_i$.*

In the Pinedo classification (Pinedo, 2016), the harness scheduling problem is $\text{Rm} \mid \text{prec, stoch} \mid \mathbb{E}[C_{\max}]$: unrelated parallel machines, precedence constraints, stochastic processing times, minimizing expected makespan.

The canonical pipeline for a GSD-style harness is a chain:

context_load $\rightarrow$ research $\rightarrow$ planning $\rightarrow$ execution $\rightarrow$ verification $\rightarrow$ review

Table 1 | Estimated stage parameters for the GSD pipeline (derived from reported ranges; no controlled measurement).

Stage	μ_i (min)	σ_i (min)	q_i	Distribution
Context loading	10	2.0	0.02	Log-normal
Research	7.5	2.5	0.05	Log-normal
Planning	6.5	1.5	0.08	Log-normal
Execution	12.5	2.5	0.15	Log-normal
Verification	10	2.0	0.10	Log-normal
Human review	10	5.0	0.05	Log-normal

2.2. Key Classical Results

[E] *Graham’s bound* (Graham, 1966): List scheduling on m identical machines achieves makespan within a factor of $(2 - 1/m)$ of optimal. For a 4-agent system, this guarantees at most $1.75\times$ optimal.

[E] *Little’s Law* (Little, 1961): $L = \lambda W$, where L is the average number of items in the system, λ is throughput, and W is average latency. This constrains the throughput–latency–concurrency triad in any pipeline.

[E] *Kingman’s approximation* (Kingman, 1961): For a GI/G/1 queue, expected wait time $\mathbb{E}[W_q] \approx \frac{c_a^2 + c_s^2}{2} \cdot \frac{\rho}{1-\rho} \cdot \mathbb{E}[S]$, where c_a^2, c_s^2 are squared coefficients of variation of interarrival and service times, and ρ is utilization. The $\rho/(1-\rho)$ term creates a “utilization cliff”: at 90% utilization, queue wait is $9\times$ that at 50%.

[E] *Amdahl’s Law* (Amdahl, 1967): If a fraction f of computation is serial, maximum speedup from parallelizing the rest is $1/f$.

3. Context Freshness and Information Decay

3.1. The Context Channel Model

Definition 3.1 (Context Fidelity). *[S] Let $\mathcal{X}$ be the true codebase state and $\mathcal{Y}$ the agent’s context representation. The context fidelity at time t after a fresh read at t_0 is:*

$$\Phi(t) = \frac{I_{\text{stale}}(t)}{I_{\text{fresh}}} = e^{-\Lambda(t-t_0)}$$

where $\Lambda = \lambda\alpha$ is the effective decay rate, λ is the codebase change rate (changes per unit time), and α is the average information disruption per change.

Theorem 3.1 (Exponential Staleness Decay). *[S] Assume: (A1) codebase changes follow a Poisson process with rate λ ; (A2) each change independently invalidates a fraction α of the agent’s cached context; (A3) α is constant across change events. Then:*

$$I_{\text{stale}}(t) = I_{\text{fresh}} \cdot e^{-\lambda\alpha(t-t_0)}$$

Remark (Assumptions and limitations). *Assumptions A2 and A3 are strong. In practice, changes are correlated (a refactor touches many files; a bugfix touches one), and the disruption fraction α varies across changes. The multi-exponential extension in Appendix B relaxes A3 by partitioning context into strata with different decay rates. The single-rate model should be understood as a first-order approximation; the sensitivity of downstream results to violations of A2–A3 is an open empirical question.*

Proof. The number of changes in $[t_0, t]$ is $N \sim \text{Poisson}(\lambda\Delta t)$ where $\Delta t = t - t_0$. After k changes, the surviving information fraction is $(1 - \alpha)^k$. Taking the expectation via the Poisson probability generating function:

$$\mathbb{E}[(1 - \alpha)^N] = e^{\lambda\Delta t((1-\alpha)-1)} = e^{-\lambda\alpha\Delta t}$$

□

The context fidelity has half-life $t_{1/2} = \ln 2 / \Lambda$.

3.2. Illustrative Calibration

Suzgun and Kalai (2024) found that fresh-context expert instances outperform accumulated-context prompting by 17.1%. This advantage likely conflates two effects: context freshness *and* task decomposition (each expert receives a narrower scope). The pure context-freshness contribution is therefore $\leq 17.1\%$; we use the full figure as an upper bound, acknowledging that the true freshness penalty may be smaller.

Treating the 17.1% as an upper bound on fidelity loss:

$$1 - \Phi_{\text{inc}} \leq 0.171 \implies \Phi_{\text{inc}} \geq 0.829$$

With a typical GSD cycle of $\bar{t} \approx 50$ minutes ≈ 0.83 hours, this gives an upper bound on the effective decay rate of $\Lambda \leq 0.226$ per hour (context half-life ≥ 3 hours). The maximum refresh interval before 20% fidelity loss is then $T_{\text{max}} \geq 59$ minutes, which aligns with the empirical GSD cycle time of 45–60 minutes. Supporting evidence (though from different experimental contexts) includes: agent drift onset at a median of 73 interactions (Anonymous, 2026), context rot affecting all 18 tested frontier models (Chroma Research, 2025), and 23% accuracy loss from just 10% irrelevant content (Chroma Research, 2025). These numbers are drawn from different systems under different conditions; they should be treated as suggestive, not as a validated calibration.

4. Verified Iterations per Hour

4.1. Definition and Decomposition

Definition 4.1 (VIH). *[S]* For a model-harness pair m operating over wall-clock time T_{wall} :

$$\text{VIH}(m) = \frac{n_{\text{verified}}(m)}{T_{\text{wall}}}$$

where n_{verified} counts only iterations passing all verification gates.

Theorem 4.1 (VIH Decomposition). *[S]* $\text{VIH} = \lambda_{\text{raw}} \cdot p_{\text{verify}}$, where $\lambda_{\text{raw}} = 1/\mathbb{E}[\tau_{\text{cycle}}]$ is the raw iteration rate and $p_{\text{verify}} = P(\text{iteration passes verification})$.

Proof. By definition, $n_{\text{verified}} = n_{\text{total}} \cdot p_{\text{verify}}$ in expectation. Since $n_{\text{total}} = \lambda_{\text{raw}} \cdot T_{\text{wall}}$, dividing by T_{wall} gives the result. $\square$

4.2. The Speed–Quality Tradeoff

Proposition 4.1 (Speed–Quality Tradeoff). *[C]* In general, p_{verify} is a decreasing function of λ_{raw} . Increasing the raw iteration rate tends to decrease the verification pass rate.

Empirical support: the DORA 2025 report finds a 7.2% delivery stability decrease per 25% AI adoption increase (Google DORA Team, 2025); GitClear reports code churn nearly doubling from 2020 to 2024 with AI adoption (GitClear, 2025); the Yerkes–Dodson law predicts performance degradation on complex tasks under time pressure (Yerkes and Dodson, 1908).

We adopt an exponential degradation model $p_{\text{verify}}(\lambda) = p_0 e^{-\beta(\lambda - \lambda_0)}$ for $\lambda \geq \lambda_0$ as a tractable parametric form. This functional form is a modeling choice, not an empirically validated relationship; the true shape of the speed–quality curve in agent pipelines is unknown. The exponential is

chosen for analytical convenience and because it satisfies the qualitative requirements (monotone decreasing, bounded between 0 and 1, convex). Results that depend on this specific functional form (particularly the closed-form $\lambda^* = 1/\beta$) should be understood as illustrative of the framework rather than quantitative predictions:

Theorem 4.2 (VIH Optimal Speed). *[S] The VIH-maximizing raw iteration rate satisfies:*

$$\frac{p'_{\text{verify}}(\lambda^*)}{p_{\text{verify}}(\lambda^*)} = -\frac{1}{\lambda^*}$$

The optimum occurs where a 1% increase in raw speed causes exactly a 1% decrease in verification rate. Under exponential degradation, $\lambda^* = 1/\beta$.

Proof. Differentiate $\text{VIH}(\lambda) = \lambda \cdot p_{\text{verify}}(\lambda)$ and set to zero: $p_{\text{verify}}(\lambda) + \lambda \cdot p'_{\text{verify}}(\lambda) = 0$. Rearranging gives the elasticity condition. Under $p'_{\text{verify}} = -\beta \cdot p_{\text{verify}}$, the condition becomes $\beta = 1/\lambda^*$. $\square$

4.3. Amdahl's Law Bound on VIH

If verification is a serial fraction f of the cycle (it runs after execution), then even with infinite parallelization of other stages (Amdahl, 1967):

$$\text{VIH}^{\max} = \frac{p_{\text{verify}}}{f \cdot \tau_{\text{cycle}}}$$

From the GSD data, $f \approx 0.20$ (verification takes 10 of 50 minutes). With $p_{\text{verify}} \approx 0.67$ (Devin's PR merge rate as proxy, Cognition AI 2025), $\text{VIH}^{\max} \approx 4.0$ verified iterations per hour, regardless of non-verification speedups.

5. Stochastic Pipeline Scheduling

5.1. Sequential Makespan

Theorem 5.1 (Sequential Makespan). *[E] For a chain pipeline under sequential execution, $\mathbb{E}[M_{\text{seq}}] = \sum_{i=1}^n \mu_i$.*

Calibrating with Table 1: $\mathbb{E}[M_{\text{seq}}] = 56.5$ minutes, consistent with the observed 45–60 minute range.

5.2. Makespan with Failures

Theorem 5.2 (Expected Makespan with Geometric Retries). *[S] Under the restart-from-failed-stage model with independent failures:*

$$\mathbb{E}[M_{\text{fail}}] = \sum_{i=1}^n \frac{\mu_i}{p_i}$$

Proof. Each stage is attempted until success; the number of attempts follows $\text{Geometric}(p_i)$ with expectation $1/p_i$. $\square$

Calibrating: $\mathbb{E}[M_{\text{fail}}] \approx 61.5$ minutes. Failures add approximately 5 minutes (8.8%) to expected cycle time.

6. Speculative Execution in Agent Pipelines

6.1. The Speculation Ratio Test

Definition 6.1 (Speculation Policy). *[S] A speculation policy $\pi : V \rightarrow \{0, 1\}$ assigns each stage a binary decision: $\pi(v_j) = 1$ if v_j begins before its predecessor commits.*

Theorem 6.1 (Speculation Ratio Test). *[S] For adjacent stages (v_i, v_j) , define the parallelism benefit $B = \mathbb{E}[\min(t_i, t_j)]$ and the rollback penalty $R = \mathbb{E}[d_i + t_i^{\text{retry}}]$. Assume R is a known scalar (i.e., rollback cost does not depend on the speculative state). Then speculation is EV-positive when:*

$$\frac{p_i}{q_i} > \frac{R}{B}$$

In practice, rollback cost in agent pipelines is state-dependent and may have a heavy tail (speculative execution can write files, trigger CI, or update caches). The test should be applied with a conservative estimate of R that accounts for worst-case rollback scenarios, not just expected cleanup time.

Proof. Expected speculative time: $\mathbb{E}[T_{\text{spec}}] = p_i \cdot \mathbb{E}[\max(t_i, t_j)] + q_i(R + \mathbb{E}[t_j])$. Sequential baseline: $\mathbb{E}[T_{\text{seq}}] = \mathbb{E}[t_i] + \mathbb{E}[t_j]$. The saving under success is $B = \mathbb{E}[t_i] + \mathbb{E}[t_j] - \mathbb{E}[\max(t_i, t_j)]$. Speculation is profitable when $p_i \cdot B > q_i \cdot R$. $\square$

At 55% next-action prediction accuracy (Zhou et al., 2025), the success-to-failure odds ratio is $0.55/0.45 = 1.22$. Speculation is profitable whenever $R/B < 1.22$, which holds when rollback cost is less than 122% of the parallelism benefit.

6.2. Speculation Depth Limit

Theorem 6.2 (Speculation Depth Limit). *[S] For a homogeneous chain with per-stage success probability p , benefit B_0 , and rollback cost R_0 , the maximum useful speculation depth is:*

$$k^* < \frac{\ln(1 + B_0/R_0)}{\ln(1/p)}$$

For $p = 0.85$ and $B_0/R_0 = 2$: $k^* \leq 6$ stages. For $p = 0.70$ and $B_0/R_0 = 1$: $k^* \leq 1$. The exponential decay of p^k places a natural horizon on useful speculation, consistent with the practical finding that deep speculation chains in agent pipelines are rarely profitable (Zhou et al., 2025).

6.3. The Spectre Analogy and Hidden Costs

Speculative execution in CPUs was considered a pure optimization for two decades before Spectre and Meltdown revealed that speculative state changes in microarchitecture created security vulnerabilities (Kocher et al., 2018). In agent pipelines, the analogous risks include: (1) state pollution from discarded speculative work, (2) combinatorial branching complexity when speculation chains interact, (3) debugging difficulty when causal chains include hypothetical branches, and (4) cognitive overhead for humans monitoring speculative pipelines. These hidden costs suggest that speculation should be applied conservatively, typically at depth 1, and only when the prediction accuracy and rollback cost satisfy the ratio test of Theorem 6.1.

7. Caching and the Staleness Tradeoff

7.1. The Cache Refresh Problem

At each step, the agent chooses between full re-read (cost R , zero staleness) and incremental update (cost $r \ll R$, staleness probability s_t). This is structurally a rent-or-buy problem (Sleator and Tarjan, 1985).

Theorem 7.1 (Cache-Staleness EOQ Theorem). *[S] If staleness grows linearly with time since last refresh (rate $\lambda\alpha$) and staleness-induced errors cost E_{stale} in expectation, the optimal refresh interval is:*

$$T^* = \sqrt{\frac{2R}{\lambda\alpha \cdot E_{\text{stale}}}}$$

This has the structure of the Economic Order Quantity formula, with R as fixed ordering cost and $\lambda\alpha \cdot E_{\text{stale}}$ as holding cost rate.

Proof. The amortized cost per unit time is $C(T) = R/T + \frac{1}{2}\lambda\alpha T \cdot E_{\text{stale}}$ (the staleness integral over $[0, T]$ under linear growth averages to $\frac{1}{2}\lambda\alpha T$). Setting $C'(T) = 0$: $-R/T^2 + \frac{1}{2}\lambda\alpha E_{\text{stale}} = 0$, yielding the stated formula. $\square$

7.2. Competitive Analysis

[E] The break-even deterministic policy (refresh after $\lceil R/r \rceil$ incremental steps) is 2-competitive against the offline optimal (Sleator and Tarjan, 1985). The optimal randomized policy achieves $e/(e-1) \approx 1.58$ -competitive (Fiat et al., 1991).

7.3. Three Caching Layers

Agent harnesses interact with three distinct caching layers, each governed by different dynamics:

KV-Cache (attention state reuse): Provider-level caching of computed key-value pairs for shared prompt prefixes. Claude Code achieves 92% hit rate and 81% cost reduction (vendor-reported; Anthropic 2026). Hit rate depends on prompt structure: append-only prompts with static prefixes maximize hits.

Prompt Caching (token-level): Server-side caching of prompt computation for repeated prefixes. Liang et al. (2026) report 41–80% cost reduction and 13–31% TTFT improvement across providers.

Plan Template Caching (reasoning-level): Reuse of structured plan templates across similar tasks. Zhang et al. (2025) achieve 50% cost and 27% latency reduction while maintaining 96.6% of optimal performance.

The Denning working set model (Denning, 1968) applies to all three layers: cache effectiveness depends on temporal locality in the request stream. The cache TTL must match the agent’s task locality window to avoid thrashing.

7.4. The Cache–Quality Tension

[C] Caching creates a fundamental tension: append-only context maximizes cache hits but accumulates noise, while context reset loses the cache but gains fresh reasoning quality (the 17.1% advantage of §3). The optimal cycle length is partially determined by when cache savings from continuation are outweighed by quality loss from context accumulation.

8. Execution Mode Selection

8.1. Decision-Theoretic Framework

The harness selects execution mode $m \in \{\text{Fast, Standard, Governed}\}$ for each task. The cost matrix $C(i, j)$ captures the asymmetric consequences of misclassification:

Chosen mode	Optimal mode		
	Fast	Standard	Governed
Fast	0	c_{FS}	c_{FG}
Standard	c_{SF}	0	c_{SG}
Governed	c_{GF}	c_{GS}	0

The critical asymmetry: $c_{FG} \gg c_{GF}$. Choosing Fast when Governed is needed risks undetected quality failures (10–100× the task time in rework), while choosing Governed when Fast suffices merely wastes time (2–5× the task time).

Theorem 8.1 (Bayesian Mode Selection). *[S] For the binary simplification (Fast vs. Governed), the optimal decision rule following Elkan (2001) is:*

$$\text{Choose Governed iff } P(\text{Governed} \mid x) > \frac{c_{GF}}{c_{FG} + c_{GF}}$$

Since $c_{FG} \gg c_{GF}$, the threshold is small (typically 0.05–0.10), meaning the harness should choose Governed even when the probability of needing it is quite low.

8.2. Bainbridge’s Ironies

Bainbridge (1983) identified a double bind in automated decision-making: if mode selection rules can be fully specified, a computer can apply them faster than a human monitor can verify; if mode selection requires judgment, automation will fail at boundary cases while the human monitor, deprived of practice, performs even worse. The implication: mode selection may require a hybrid approach where the harness proposes a mode and the human confirms or overrides, rather than full automation. The feature engineering problem is itself recursive: deciding how much analysis to do requires analysis.

9. The Speed–Quality Pareto Frontier

9.1. Frontier Structure

Definition 9.1 (Pareto Frontier). *[E] Let C be the set of harness configurations. The Pareto frontier $\mathcal{F} = \{c \in C : \nexists c' \text{ with } S(c') \geq S(c), Q(c') \geq Q(c), \text{ and at least one strict}\}$.*

Proposition 9.1 (Mixed Frontier Shape). *[C] The empirical Pareto frontier has three regimes: (1) concave at low speed (redundant checks; easy to gain speed cheaply), (2) convex at moderate speed (each gain requires skipping progressively more important verification), and (3) concave again at high speed (statistical filtering via best-of- N stabilizes quality).*

Evidence for Regime 3 comes from SWE-bench data showing Pass@5 substantially exceeding Pass@1, meaning running 5× more cheap iterations with majority voting yields higher effective quality than fewer careful iterations (Anonymous, 2025).

9.2. Mixing Strategies

Theorem 9.1 (Convexification). *[E+S] In concave frontier regions, the mixed strategy using configuration c_1 with probability α and c_2 with probability $(1-\alpha)$ achieves $(S_{\text{mix}}, Q_{\text{mix}}) = \alpha(S_1, Q_1) + (1-\alpha)(S_2, Q_2)$, which Pareto-dominates every pure strategy in that region.*

This is the direct analogue of the classical result that mixed strategies expand the achievable set to the convex hull of pure-strategy outcomes.

9.3. VIH Maximization

Theorem 9.2 (VIH Tangency Condition). *[S] $\text{VIH} = S \cdot Q$ is maximized on the frontier where the iso-VIH hyperbola $Q = \text{VIH}/S$ is tangent to the frontier curve $Q = f(S)$. At this point, the elasticity of quality with respect to speed is exactly -1 :*

$$\epsilon_{Q,S} = \frac{dQ}{dS} \cdot \frac{S}{Q} = -1$$

Below this point, speed gains improve VIH; above it, speed gains degrade VIH.

Proof. Maximize $\text{VIH}(S) = S \cdot f(S)$. Setting $\text{VIH}'(S) = f(S) + S f'(S) = 0$ gives $f'(S^*) = -f(S^*)/S^* = -Q^*/S^*$, which is the elasticity condition. $\square$

10. Illustrative Calibration

The calibration in this section draws on published data from multiple systems (GSD, Claude Code, SWE-bench, Devin) under different conditions. No single system has been measured end-to-end against the model’s predictions. The stage parameters in Table 1 are estimates derived from reported ranges, not from controlled measurement with known sample sizes. The results should be understood as illustrative of the framework’s predictions, not as validated empirical findings.

10.1. Pipeline Speedup Analysis

We estimate the expected speedup from four proposed strategies against the GSD baseline of $\mathbb{E}[T_{\text{seq}}] = 56.8$ minutes:

Table 2 | Speedup analysis for the four temporal optimization strategies.

Strategy	Time saved	Quality impact	VIH impact	Net verdict
Persistent state	7.0 min	−17.1%	Negative	VIH-negative
Speculative pipelining	8.5 min	Minimal	+19%	Positive
Tiered verification	4.8 min	−1.5%	+7.6%	Positive
Async governance	8.0 min	Minimal	+17%	Positive
Combined (VIH-optimal)	19.3 min	−1.5%	+34%	Positive

The key finding: **naive persistent state is likely VIH-negative** when the quality penalty approaches the 17.1% upper bound. The break-even threshold (the ratio of time saved to total cycle time) is approximately 12.3% for these parameters, though this threshold is parameter-specific and shifts with cycle time and time savings.

The combined speedup from speculative pipelining, tiered verification, and asynchronous governance is estimated at $\mathbb{E}[T_{\text{combined}}] \approx 37.5$ minutes. This estimate assumes approximate additivity of the three optimizations, which is a simplification: in practice, speculative pipelining accuracy depends on verification depth, and async governance changes the pipeline structure. The true combined speedup may be smaller due to interaction effects. A conservative estimate would be 20–34% cycle time reduction.

10.2. Token Economics

Production agent workflows exhibit input-to-output token ratios of approximately 100:1 (Manus AI, 2025). With the 10× price differential between cached (\$0.30/MTok) and uncached (\$3.00/MTok) input tokens, caching is economically decisive. At Claude Code’s 92% hit rate, total cost reduction is 78.9%, bounded at 85.7% because output tokens are irreducible.

The VIH-adjusted cost per verified iteration is:

$$C_{\text{verified}} = \frac{C_{\text{iteration}}}{p_{\text{verify}}}$$

The “token snowball” effect (Anonymous, 2025) means failed attempts consume 3–13× more tokens than successful ones, making failure the dominant cost driver. A 1.71× more expensive model can be cheaper overall if it raises success rate from 27% to 40%.

10.3. Diminishing Returns in Multi-Agent Iteration

Multi-agent code verification shows diminishing returns: +14.9 pp, +13.5 pp, +11.2 pp per additional agent. Fitting a saturating exponential $Q(n) = Q_{\text{max}}(1 - e^{-an})$, the VIH-optimal agent count is approximately $k^* = 4$.

11. Related Work

Agent efficiency benchmarks. SWE-bench (Anonymous, 2025) measures resolve rates, but SWE-Effi introduces resource-bounded effectiveness scores, penalizing high-scoring but slow agents. Bian et al. (2025) decompose bottlenecks, finding web environment latency dominates (53.7% of runtime) with LLM API latency varying up to 69×.

Speculative execution for agents. Zhou et al. (2025) achieve 46–55% next-action prediction accuracy. Ji et al. (2026) extend this with heterogeneous speculation for research agents, achieving 3.28× speedup. Our contribution is the formal speculation ratio test and depth limit, grounded in classical scheduling theory.

Caching for LLM agents. Liang et al. (2026) evaluate prompt caching empirically. Zhang et al. (2025) introduce plan template caching at the reasoning level. Our contribution is the competitive-analysis grounding and EOQ-analog theorem.

Context management. Suzgun and Kalai (2024) establish the Fresh Eyes advantage. Liu et al. (2024) identify the “lost in the middle” phenomenon. Anonymous (2026) quantify performance degradation over extended interactions. Our contribution is the information-theoretic formalization connecting these empirical findings.

Speed–quality tradeoffs in software. Reinertsen’s *Principles of Product Development Flow* (2009) formalizes the cost-of-delay framework and batch-size optimization for product development, providing the direct ancestor of our VIH-based approach in a more general setting. Forsgren et al.

(2018) show that speed and stability can be improved simultaneously through DevOps practices. The DORA 2025 report (Google DORA Team, 2025) complicates this picture for AI-assisted development, finding throughput gains but stability losses. Kahneman (2011) provides the cognitive science foundation for why speed pressure degrades complex reasoning.

12. Design Principles

The formal analysis yields six actionable principles:

1. **Optimize VIH, not raw speed.** $VIH = \lambda_{\text{raw}} \cdot p_{\text{verify}}$ captures the speed–quality tradeoff in a single metric. Monitor it directly.
2. **Speculate conservatively.** Use the ratio test ($p/q > R/B$) to decide whether to speculate. Limit speculation depth to 1 for pipelines with per-stage success probability below 85%. The Spectre analogy warns that hidden costs may not surface immediately.
3. **Refresh context at the EOQ interval.** The optimal refresh interval $T^* = \sqrt{2R/(\Lambda \cdot E_{\text{stale}})}$ balances re-read cost against staleness risk. For typical parameters, this is 15–60 minutes.
4. **Bias mode selection toward safety.** The asymmetric cost structure ($c_{\text{FG}} \gg c_{\text{GF}}$) means choosing Governed mode at even 5–10% posterior probability of needing it is Bayes-optimal.
5. **Exploit mixing in concave frontier regions.** When two operating points span a concave region of the speed–quality frontier, probabilistic alternation dominates any intermediate pure strategy.
6. **Treat the 12.3% threshold as a gate for persistent state.** Any context reuse strategy must demonstrate quality degradation below 12.3% (the break-even for a 7-minute time saving on a 57-minute cycle) to be VIH-positive.

13. Contrarian Positions and Limitations

We engage seven contrarian positions that challenge the temporal optimization thesis.

“Speed kills quality.” The Yerkes–Dodson law (Yerkes and Dodson, 1908) predicts that complex tasks degrade under time pressure. Incubation effects in creative problem-solving (Sio and Ormerod, 2009; Wallas, 1926) suggest that pauses between work sessions do genuine cognitive work. GitClear data showing doubled code churn with AI adoption (GitClear, 2025) supports this concern. The strongest form of this argument is that *verification itself becomes less reliable under time pressure*, creating a feedback loop: faster iterations produce lower quality, and the verification that would catch the quality loss is itself degraded. If verification fidelity degrades with speed, VIH is systematically overestimated at high speeds, because p_{verify} as measured by the harness diverges from the true pass rate. Our framework does not account for this feedback loop, and doing so would require a model of verification accuracy as a function of speed, which remains an open empirical question.

“Fresh Eyes is worth the cost.” The 17.1% advantage is large. Our analysis shows that naive persistent state is indeed VIH-negative (§10). The contrarian position is partially validated; incremental context is only justified when compaction reduces quality loss below 12.3%.

“Speculative execution has hidden costs.” The Spectre analogy (Kocher et al., 2018) is apt: speculative state changes may have consequences that do not surface for a long time. We limit speculation depth formally (Theorem 6.2), but the complexity budget consumed by speculation management competes with other engineering needs. For many workloads, the 10–30% cycle time saving may not justify the implementation and debugging complexity.

“Mode selection is hard to automate.” Bainbridge’s ironies (Bainbridge, 1983) create a genuine double bind. Our Bayesian framework (Theorem 8.1) provides a principled threshold, but the features needed for accurate classification (semantic impact, architectural coupling) are themselves expensive to compute, creating a recursive problem.

“Caching is a false economy.” Cache invalidation bugs are intermittent, state-dependent, and compound through cascading cached values. For rapidly changing codebases (active development), cache hit rates may be too low to justify the infrastructure overhead. Our EOQ theorem (Theorem 7.1) provides a principled refresh schedule, but assumes knowledge of the change rate λ , which itself must be estimated.

“VIH is the wrong metric.” The most fundamental objection: VIH treats all verified iterations as equally valuable, but they are not. A verified iteration that introduces subtle architectural debt, passes tests but degrades maintainability, or solves the wrong subproblem is “verified” but not valuable. The correct target may be *value delivered per unit cost*, incorporating developer time, compute cost, architectural debt, and opportunity cost. VIH is a useful proxy for the speed–quality tradeoff specifically, but it should not be mistaken for a complete objective function. Reinertsen’s cost-of-delay framework provides a more comprehensive alternative that accounts for the economic value of delivery timing.

Limitations. Our formal models make simplifying assumptions: Poisson change processes, exponential quality degradation, independence of stage durations, and log-normal stage distributions. The context fidelity model treats staleness as uniform across information types, while in reality, architectural knowledge decays slowly and implementation details decay quickly. The VIH metric, while better than raw speed, does not capture all quality dimensions relevant to long-lived software systems. Empirical validation through controlled experiments (outlined in Appendix A) is needed to test the key predictions.

14. Conclusion

We have developed a formal framework for the temporal architecture of AI coding agent harnesses, grounded in classical scheduling, queueing, speculative execution, and cache theory. The central insight is that a useful intermediate metric, Verified Iterations per Hour, has a non-trivial maximum: pushing speed beyond the unit-elasticity point on the speed–quality Pareto frontier degrades effective throughput. The framework yields concrete, falsifiable predictions: naive persistent state is VIH-negative when quality penalties exceed a cycle-time-dependent threshold; speculation is EV-positive only when the success-to-failure odds ratio exceeds the rollback-to-benefit ratio; the optimal cache refresh interval follows an EOQ-analog formula; and mode selection should be biased toward safety by the cost asymmetry ratio. Illustrative calibration suggests that speculative pipelining, tiered verification, and asynchronous governance can reduce GSD cycle time by 20–34% without significant quality degradation. VIH is not a complete objective function (it ignores failure severity, architectural debt, and delivery value), but it formalizes the speed–quality tradeoff that currently dominates harness design decisions. Controlled experiments validating the key predictions (Appendix A) are needed before the quantitative results should be treated as more than illustrative.

Appendices

A. Verification Roadmap

We outline five experiments to validate the key theoretical predictions:

Experiment 1: VIH is maximized at intermediate speed. Run 100 identical coding tasks at varying speed levels (by adjusting model temperature, planning depth, and verification thoroughness). Measure VIH at each level. Prediction: VIH peaks at an intermediate speed and declines at maximum throughput.

Experiment 2: Speculation EV threshold. Implement speculative pipelining with varying prediction accuracy (by injecting controlled noise into the predictor). Measure cycle time with and without speculation. Prediction: net cycle time reduction is positive only when prediction accuracy exceeds $R/(R+B)$.

Experiment 3: Cache refresh interval. Vary the refresh interval from 5 to 120 minutes across 50 coding sessions. Measure error rates and total cost. Prediction: optimal interval follows $T^* \propto \sqrt{R/(\lambda \cdot E_{\text{stale}})}$.

Experiment 4: Mode misclassification asymmetry. Record 200 mode selection decisions with ground-truth labels (determined post hoc by senior engineers). Compute costs of Type I and Type II errors. Prediction: Type II costs (Fast when Governed needed) exceed Type I costs (Governed when Fast suffices) by 10× or more.

Experiment 5: Context fidelity decay. Measure task accuracy at 10, 30, 60, and 120 minutes after the last full context read, controlling for task difficulty. Prediction: accuracy decays exponentially with time, with half-life approximately 3 hours.

B. Extended Proofs

B.1. Multi-Exponential Context Decay

The single-rate model of Theorem 3.1 can be extended to K information strata with different disruption rates:

$$I_{\text{stale}}(t) = \sum_{k=1}^K I_k \cdot e^{-\lambda \alpha_k (t-t_0)}$$

where $\sum_k I_k = I_{\text{fresh}}$. This captures the empirical observation that architectural knowledge (α_k small) persists longer than implementation details (α_k large). The aggregate fidelity is a weighted sum of exponentials, which is no longer a single exponential but retains the monotone decreasing property.

B.2. VIH with Rework Cycles

If failed iterations require r rework cycles on average:

$$\text{VIH}_{\text{eff}} = \frac{\lambda_{\text{raw}} \cdot p_{\text{verify}}}{1 + (1 - p_{\text{verify}}) \cdot r}$$

For $r > 0$, this penalizes low p_{verify} more severely than the basic VIH formula. The optimal speed under rework is strictly lower than under the no-rework model.

B.3. Sequential Mode Escalation via SPRT

Instead of a one-shot mode decision, the harness can use Wald’s Sequential Probability Ratio Test to accumulate evidence and escalate from Fast to Governed as the task reveals its complexity. The test

continues sampling features until the likelihood ratio $\Lambda_n = \prod_{i=1}^n f_1(x_i)/f_0(x_i)$ crosses either threshold $A = (1 - \beta)/\alpha$ (escalate to Governed) or $B = \beta/(1 - \alpha)$ (commit to Fast), where α is the false-escalation rate and β is the missed-escalation rate. This provides an adaptive alternative to the one-shot Bayesian classifier of Theorem 8.1.

References

- G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30:483–485, 1967.
- Anonymous. SWE-effi: Resource-aware effectiveness metrics for software engineering agents. *arXiv preprint arXiv:2509.09853*, 2025.
- Anonymous. Agent drift: Quantifying performance degradation in long-running agent systems. *arXiv preprint arXiv:2601.04170*, 2026.
- Anthropic. Agentic coding trends report 2026. <https://resources.anthropic.com/2026-agentic-coding-trends-report>, 2026.
- L. Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983.
- L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- N. Bian et al. What limits agentic systems efficiency? *arXiv preprint arXiv:2510.16276*, 2025.
- Chroma Research. Context rot: Why LLMs degrade as context grows. <https://research.trychroma.com/context-rot>, 2025.
- Cognition AI. Devin annual performance review 2025. <https://cognition.ai/blog/devin-annual-performance-review-2025>, 2025.
- P. J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.
- C. Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence*, pages 973–978, 2001.
- A. Fiat, R. M. Karp, M. Luby, L. A. McGeoch, D. D. Sleator, and N. E. Young. Competitive paging algorithms. *Journal of Algorithms*, 12(4):685–699, 1991.
- N. Forsgren, J. Humble, and G. Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.
- GitClear. AI copilot code quality: 2025 data. https://www.gitclear.com/ai_assistant_code_quality_2025_research, 2025.
- Google DORA Team. Accelerate state of DevOps report 2025. <https://dora.dev/research/2025/dora-report/>, 2025.
- R. L. Graham. Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45(9):1563–1581, 1966.
- W. J. Hopp and M. L. Spearman. *Factory Physics*. McGraw-Hill, 2nd edition, 2000.

- Iskohm. AI agents comparison 2026. <https://blog.iskohm.com/en/posts/ai-agents-comparison-2026/>, 2026.
- Y. Ji et al. DualSpec: Heterogeneous speculation for deep research agents. *arXiv preprint arXiv:2603.07416*, 2026.
- D. Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.
- J. F. C. Kingman. The single server queue in heavy traffic. *Mathematical Proceedings of the Cambridge Philosophical Society*, 57(4):902–904, 1961.
- P. Kocher et al. Spectre attacks: Exploiting speculative execution. <https://spectreattack.com/spectre.pdf>, 2018.
- Z. Liang et al. Don’t break the cache: An evaluation of prompt caching for long-horizon agentic tasks. *arXiv preprint arXiv:2601.06007*, 2026.
- J. D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961.
- N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- Manus AI. Context engineering for AI agents: Lessons from building Manus. <https://manus.im/blog/Context-Engineering-for-AI-Agents>, 2025.
- R. H. Möhring, F. J. Radermacher, and G. Weiss. Stochastic scheduling problems I: General strategies. *Zeitschrift für Operations Research*, 28:193–260, 1984.
- M. L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 5th edition, 2016.
- U. N. Sio and T. C. Ormerod. Does incubation enhance problem solving? A meta-analytic review. *Psychological Bulletin*, 135(1):94–120, 2009.
- D. D. Sleator and R. E. Tarjan. Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28(2):202–208, 1985.
- J. E. Smith. A study of branch prediction strategies. In *Proceedings of the 8th International Symposium on Computer Architecture*, pages 135–148, 1981.
- M. Suzgun and A. T. Kalai. Meta-prompting: Enhancing language models with task-agnostic scaffolding. *arXiv preprint arXiv:2401.12954*, 2024.
- R. M. Tomasulo. An efficient algorithm for exploiting multiple arithmetic units. *IBM Journal of Research and Development*, 11(1):25–33, 1967.
- G. Wallas. *The Art of Thought*. Harcourt, Brace and Company, 1926.
- T.-Y. Yeh and Y. N. Patt. Two-level adaptive training branch prediction. In *Proceedings of the 24th International Symposium on Microarchitecture*, pages 51–61, 1991.
- R. M. Yerkes and J. D. Dodson. The relation of strength of stimulus to rapidity of habit-formation. *Journal of Comparative Neurology and Psychology*, 18(5):459–482, 1908.
- Y. Zhang et al. Agentic plan caching: Test-time memory for fast and cost-efficient LLM agents. In *NeurIPS*, 2025.
- D. Zhou et al. Speculative actions: A lossless framework for faster agentic systems. *arXiv preprint arXiv:2510.04371*, 2025.